

MICROPROCESSOR & EMBEDDED SYSTEMS

8086 Architecture · Instruction Set · Assembly Programming

Unit	Coverage	Weight
Unit I	8086 Architecture, Addressing, Instructions, ALPs	■■■ HIGH
Unit II	Modular Prog, Data Conversion, DOS/BIOS, ARM	Medium
Unit III	Hardware, Machine Code, Interrupts	Medium

■ MASTER EXAM GUIDE — All PPTs + Question Bank Analysed

Covers: Architecture · Flag Register · Segment Registers · Addressing Modes · Instruction Set · BCD/ASCII Arithmetic · ALPs · Assembler Directives

CHAPTER 1: SUBJECT OVERVIEW & EXAM STRATEGY

1.1 Important Units & Weightage

Topic / Sub-topic	Questions in QB	Priority
8086 Architecture (BIU, EU, Registers)	5+	■ MUST DO
Flag Register (all 9 flags)	4+	■ MUST DO
Segment Registers & Physical Address	6+	■ MUST DO
Addressing Modes	5+	■ MUST DO
Instruction Set (MOV,PUSH,POP,IN,XCHG,MUL)	6+	■ MUST DO
Shift & Rotate Instructions	4+	■ MUST DO
BCD/ASCII Instructions (DAA, AAA, AAM)	4+	■ HIGH
Assembler Directives	4+	■ HIGH
ALP Programs (search, palindrome, string)	5+	■ MUST DO
Logical Instructions (AND,OR,XOR,NOT)	2+	■ MEDIUM

1.2 Most Scoring Topics (Score 30+ marks here)

- 8086 Block Diagram with explanation (10 marks – appears every exam)
- Physical Address calculation – all 4 addressing modes (10 marks)
- Flag register neat diagram + all 9 flags explained (6–8 marks)
- ALP for linear search / palindrome / string (8 marks)
- Shift & Rotate instructions with diagrams (8 marks)
- Assembler Directives ASSUME, DB, EQU, MACRO, SEGMENT (4–8 marks)
- DAA and AAA with worked examples (6–8 marks)

1.3 Exam Strategy

- **Study Order:** Architecture → Flag Register → Segment Registers → Addressing Modes → Instruction Set → Shift/Rotate → BCD Arith → Directives → ALPs
- **Safe skips (if time is very short):** AAD, AAS, DAS, IDIV internals
- **Always draw** the 8086 block diagram even if not asked—partial credit!
- **For ALPs:** write the logic in English first, then translate to assembly.
- **Common mistake:** confusing logical address (segment:offset) with physical address.
- **Physical address formula:** $PA = \text{Segment_Register} \times 10H + \text{Offset (effective address)}$

CHAPTER 2: 8086 ARCHITECTURE

2.1 Big Picture – What is 8086?

The Intel 8086 is a 16-bit microprocessor introduced in 1978 (3rd generation). It has a **20-bit address bus** (can address up to 1 MB of memory) and a **16-bit data bus**.

Key Numbers to Remember

Property	Value
Data bus width	16 bits
Address bus width	20 bits
Addressable memory	2^20 = 1 MB (1,048,576 locations)
Internal registers	14 registers (16-bit each)
Instruction queue	6 bytes (prefetch queue)
Technology	HMOS (3rd gen)
Clock frequency	5–10 MHz (original 8086)

2.2 Two-Unit Architecture: BIU and EU

The 8086 is internally divided into two independent units that work in parallel (pipeline):

Bus Interface Unit (BIU)	Execution Unit (EU)
Fetches instructions from memory into the 6-byte instruction queue	Decodes and executes instructions from the queue
Contains: Segment Registers (CS, DS, SS, ES), IP, Instruction Counter	Contains: ALU, Control Unit, Flag Register, General-purpose registers
Manages all memory/I/O read-write operations on the bus	Performs arithmetic, logic, shift operations on data
Sends 20-bit physical address to memory	Sends EA (effective address) to BIU for memory access

■ PIPELINING: While EU executes the current instruction, BIU fetches the next instruction. This overlapping speeds up execution significantly!

2.3 Registers – Complete Reference

A) General-Purpose Registers (Data Register File)

Register	Full Name	Special Uses
AX (AH+AL)	Accumulator	MUL, DIV, IN, OUT results; 32-bit = DX:AX
BX (BH+BL)	Base Register	Memory addressing ([BX]); only GPR used as pointer

CX (CH+CL)	Count Register	Loop counter (LOOP inst), shift count (CL)
DX (DH+DL)	Data Register	32-bit MUL/DIV (DX:AX); 16-bit I/O port address in DX

B) Pointer & Index Registers

Register	Name	Default Segment	Use
SP	Stack Pointer	SS	Points to top of stack; auto inc/dec with PUSH/POP
BP	Base Pointer	SS	Access stack frame parameters; can also use DS
SI	Source Index	DS	Source in string operations; index addressing
DI	Destination Index	ES (strings) / DS	Dest in string ops; index addressing
IP	Instruction Pointer	CS	Offset of next instruction; auto updated by BIU

C) Segment Registers

Register	Segment	Holds Address of...
CS	Code Segment	Currently executing program instructions
DS	Data Segment	Program data (variables, arrays)
SS	Stack Segment	The stack (PUSH/POP operations)
ES	Extra Segment	Extra data; destination in string operations

2.4 Physical Address Generation (VERY IMPORTANT!)

Since segment registers are 16-bit but the address bus is 20-bit, the BIU generates a 20-bit physical address as follows:

Physical Address = Segment Register × 10H + Effective Address (Offset)

OR equivalently: $PA = (\text{Segment Register} \ll 4) + \text{Offset}$

Example: DS = 2000H, BX = 3FFFH, DI = 5A00H

Instruction: ADD [BX+DI], CL → Based Indexed Addressing

$EA = BX + DI = 3FFFH + 5A00H = 99FFH$

$PA = DS \times 10H + EA = 20000H + 99FFH = \mathbf{299FFH}$

Worked Examples from Question Bank:

Instruction	Mode	EA Calculation	PA (DS=2000H)
ADD [DI], AL	Register Indirect	$EA = DI = 5A00H$	$2000H \times 10H + 5A00H = 25A00H$
SUB BLOCK, AX	Direct	$EA = \text{BLOCK} = 4000H$	$2000H \times 10H + 4000H = 24000H$
AND [BX+DI], CL	Based Indexed	$EA = BX + DI = 3FFF + 5A00 = 99FFH$	$20000H + 99FFH = 299FFH$

ADC [BP+100H], AX	Based with Disp (SS)	EA = BP+100H = 69B0+100 = 6AB0H	SS×10H+6AB0H = 46AB0H
-------------------	----------------------	---------------------------------	-----------------------

■■ NOTE: [BP] uses SS by default. All others use DS by default.

CHAPTER 3: FLAG REGISTER

3.1 Flag Register Layout (16-bit)

The 8086 has a 16-bit flag register. Only 9 of the 16 bits are used. Bits 1, 3, 5, 12, 13, 14, 15 are undefined.

Bit 15	14	13	12	11 OF	10 DF	9 IF	8 TF	7 SF	6 ZF	5	4 AF	3	2 PF	1	0 CF
—	—	—	—	O	D	I	T	S	Z	—	A	—	P	—	C

Red = Control flag (OF)

Green = Status flags

Amber = Control flags (D,I,T)

3.2 Status Flags (Set by arithmetic/logic instructions)

Flag	Bit	Mnemonic	Set to 1 when...	Example
Carry	0	CF	Carry/borrow out of MSB in arithmetic	FFH + 01H → CF=1
Parity	2	PF	Lower 8 bits of result have even number of 1s	0000 0011 → PF=1 (two 1s)
Aux Carry	4	AF	Carry from bit 3 to bit 4 (used in BCD)	BCD adjustment uses AF
Zero	6	ZF	Result of operation is zero	5 - 5 = 0 → ZF=1
Sign	7	SF	MSB of result is 1 (negative in signed)	-5 in 2's comp → SF=1
Overflow	11	OF	Signed arithmetic overflow (result too large)	7FH + 01H = 80H → OF=1

3.3 Control Flags (Set by programmer to control processor behaviour)

Flag	Bit	Effect when SET (=1)	Effect when CLEARED (=0)	Set/Clear Instruction
Direction (DF)	10	String ops auto-DECREMENT SI, DI (process backwards)	String ops auto-INCREMENT SI, DI (process forwards)	STOS (set) / CWD (clear)
Interrupt (IF)	9	External maskable interrupts (INT) ENABLED	External maskable interrupts DISABLED	STI (set) / CLI (clear)
Trap (TF)	8	Single-step mode: INT 1 after every instruction (debug)	No instruction (no debug)	Set/clear via PUSHF/POPF

Memory Trick (Flags): "O D I T S Z A P C"

→ Oh Dear, I Think Stephanie Zargaryan Ate Pineapple Cake

Status flags: CF, PF, AF, ZF, SF, OF | Control flags: DF, IF, TF

CHAPTER 4: ADDRESSING MODES

4.1 What is an Addressing Mode?

An addressing mode specifies **how the operand of an instruction is located/accessed**. It tells the CPU where to find the data.

8086 addressing modes are divided into: (1) Data Addressing Modes (2) Branch Addressing Modes

Key Formula: Effective Address (EA) = Base + Index + Displacement

Mode	EA Formula	Example	Registers Allowed
Immediate	Operand IS the data (no EA)	MOV AX, 1234H	— (data in instruction)
Register	Operand is in a register	MOV AX, BX	AX,BX,CX,DX,SI,DI,SP,BP
Direct	EA = Displacement (16-bit)	MOV AX, [2000H]	Any 16-bit offset
Register Indirect	EA = [Reg]	MOV AX, [BX]	BX, SI, DI, BP
Based w/ Disp	EA = Base + Disp	MOV AX, [BX+10H]	BX or BP
Indexed w/ Disp	EA = Index + Disp	MOV AX, [SI+20H]	SI or DI
Based Indexed	EA = Base + Index	MOV AX, [BX+SI]	BX/BP + SI/DI
Based Indexed+Disp	EA = Base + Index + Disp	MOV AX, [BX+DI+100H]	BX/BP + SI/DI + any disp

RULES to Remember:

- Base registers: BX or BP only
- Index registers: SI or DI only
- [BP] uses SS segment by default (all others use DS)
- You CANNOT combine two base or two index registers: [BX+BP] is INVALID, [SI+DI] is INVALID
- Direct mode: the 16-bit number in [] is the offset directly

4.2 I/O Port Addressing

Type	Port Address Range	Instruction Format	Example
Fixed/Direct Port	00H to FFH (8-bit)	IN AL, port# / OUT port#, AL	IN AL, 0C8H
Variable/Indirect Port	0000H to FFFFH (16-bit)	IN AL, DX / OUT DX, AL	MOV DX,0FF78H / IN AL,DX

■ Only AL (byte) or AX (word) can be used with IN/OUT. Port 0–FFH can be direct; 0–FFFFH needs DX.

CHAPTER 5: 8086 INSTRUCTION SET

5.1 Data Transfer Instructions

These instructions copy data between registers, memory, and I/O ports. **They do NOT affect flags** (except POPF).

Instruction	Syntax	Operation	Key Point
MOV	MOV dst, src	dst ← src (copy)	Cannot MOV mem to mem; cannot MOV immediate to seg reg directly
XCHG	XCHG dst, src	swap dst ↔ src	Both operands must be same size; no flags affected
PUSH	PUSH src	SP ← SP - 2; [SS:SP] ← src	SP decrements by 2 (word pushed); src can be reg or mem
POP	POP dst	dst ← [SS:SP]; SP ← SP + 2	SP increments by 2; cannot POP CS
IN	IN AL/AX, port	AL/AX ← port data	port can be 8-bit immediate or DX (16-bit)
OUT	OUT port, AL/AX	port ← AL/AX	Same port rules as IN
CBW	CBW	Sign-extend AL → AX	AH = 00H if AL positive, FFH if negative
CWD	CWD	Sign-extend AX → DX:AX	Useful before IDIV (word)

MOV Restrictions (Common Exam Question!):

- ✗ MOV BL, AX (size mismatch – byte vs word)
- ✗ MOV DS, 5000H (cannot move immediate directly to segment register)
- ✗ ADD [5000H], [3000H] (two memory operands not allowed)
- ✗ OUT 9000H, AX (port > FFH must use DX)
- ✓ ADD AX, 100H (valid – register + immediate)
- ✓ ADD CX, 1234H (valid)

5.2 Arithmetic Instructions

Instruction	Operation	Flags Affected	Example
ADD dst, src	dst = dst + src	OF, SF, ZF, AF, PF, CF	ADD AX, BX
ADC dst, src	dst = dst + src + CF	All above	ADC AX, 0 (add carry)
SUB dst, src	dst = dst - src	OF, SF, ZF, AF, PF, CF	SUB CX, 1
SBB dst, src	dst = dst - src - CF	All above	SBB AX, DX
CMP src1, src2	src1 - src2, flags set, result discarded	All above	CMP AX, 24H
MUL src	AX = AL × src (byte); DX:AX = AX × src (word)	OF, CF	MUL BL
IMUL src	Signed version of MUL	CF, OF	IMUL BH
DIV src	AL = AX ÷ src, AH = rem (byte); AX = DX:AX ÷ src, DX = rem	OF, DF, IF, OF	DIV CL
IDIV src	Signed version of DIV	Undefined	IDIV BL

INC dst	dst = dst + 1	OF,SF,ZF,AF,PF (not CF!)	INC CX
DEC dst	dst = dst - 1	OF,SF,ZF,AF,PF (not CF!)	DEC BX
NEG dst	dst = 0 - dst (2's complement)	All	NEG AL

■■ INC and DEC do NOT affect the Carry Flag (CF). This is intentional so loop counters do not disturb carry.

5.3 Logical Instructions

Instruction	Operation	Use Case	Example
AND dst, src	dst = dst AND src (bit-by-bit)	Masking (clear specific bits)	AND AL, 0FH (clear upper nibble)
OR dst, src	dst = dst OR src	Setting specific bits	OR AL, 80H (set MSB)
XOR dst, src	dst = dst XOR src	Toggle bits; clear reg (XOR AX, AX)	XOR AX, AX → AX=0
NOT dst	dst = NOT dst (1's comp)	Invert all bits	NOT AL
TEST src1,src2	src1 AND src2 (flags only, no change)	Check specific bits w/o changing	TEST AL, 01H (check bit 0)

5.4 Shift & Rotate Instructions (HIGH PRIORITY!)

Key Concepts:

- Shift LEFT by 1 bit = Multiply by 2
- Shift RIGHT by 1 bit = Divide by 2
- Count = 1 → write directly; Count > 1 → load into CL register
- SHL and SAL are identical; SHR is logical (0 in MSB); SAR is arithmetic (sign extends)

Instruction	Action	CF gets...	New bit filled with...	Use
SHL / SAL	Shift bits LEFT	Old MSB	0 in LSB	Multiply unsigned/signed × 2
SHR	Shift bits RIGHT	Old LSB	0 in MSB	Divide UNSIGNED by 2
SAR	Shift bits RIGHT (arithmetic)	Old LSB	Sign bit (MSB copy)	Divide SIGNED by 2
ROL	Rotate LEFT (circular)	Old MSB	Old MSB → LSB	Bit manipulation
ROR	Rotate RIGHT (circular)	Old LSB	Old LSB → MSB	Bit manipulation
RCL	Rotate LEFT through CF	Old MSB	Old CF → LSB	9-bit/17-bit rotate with carry
RCR	Rotate RIGHT through CF	Old LSB	Old CF → MSB	9-bit/17-bit rotate with carry

```
Example: SHR AX, 1      ; divide AX by 2
          MOV CL, 3
          SHL BX, CL    ; multiply BX by 8 (2^3)
          SAR AL, 1     ; signed divide AL by 2 (preserves sign)
```

Difference: Logical vs Arithmetic Shift RIGHT

SHR: 1000 0000 → 0100 0000 (MSB becomes 0, treats as unsigned)

SAR: 1000 0000 → 1100 0000 (MSB STAYS 1, preserves negative sign)

CHAPTER 6: BCD & ASCII ARITHMETIC

6.1 BCD Basics

BCD (Binary Coded Decimal) stores each decimal digit in 4 bits.

Packed BCD: Two decimal digits per byte. E.g., 59 decimal = 0101 1001 = 59H (packed)

Unpacked BCD: One decimal digit per byte (upper nibble = 0). E.g., 5 = 0000 0101

Instruction	Type	Use After	What it does
DAA	Packed BCD	ADD	Adjusts AL to correct packed BCD after ADD. Adds 6 to nibble if >9 or carry
DAS	Packed BCD	SUB	Adjusts AL to correct packed BCD after SUB. Subtracts 6 if needed
AAA	Unpacked BCD	ADD	Adjusts AL to unpacked BCD after ADD. Increments AH if carry
AAS	Unpacked BCD	SUB	Adjusts AL to unpacked BCD after SUB. Decrements AH if borrow
AAM	Unpacked BCD	MUL	Adjusts AX to two unpacked BCD digits after MUL. Divides by 10
AAD	Unpacked BCD	DIV (before)	Converts two unpacked BCD digits in AX to binary BEFORE division

6.2 DAA – Decimal Adjust after Addition (EXAM FAVOURITE!)

Rule: After ADD AL, src:

- If (AL AND 0FH) > 9 OR AF = 1 → Add 06H to AL
- If (new AL upper nibble) > 9 OR CF = 1 → Add 60H to AL, set CF

Example 1: 59 BCD + 35 BCD

```
AL = 59H, BL = 35H
ADD AL, BL → AL = 8EH
DAA          → Lower nibble E > 9, so add 06H
              AL = 94H (94 BCD) ✓
```

Example 2: 88 BCD + 49 BCD

```
AL = 88H, BL = 49H
ADD AL, BL → AL = D1H, AF = 1
DAA          → AF=1, add 06H → AL = D7H
              Upper nibble D > 9, add 60H → AL = 37H, CF = 1
              Result: 137 BCD (CF=1 means hundreds carry)
```

6.3 AAA – ASCII Adjust after Addition

Used with ASCII/unpacked BCD addition. ASCII digits: 0=30H, 1=31H ... 9=39H

```
Example: ASCII 5 + ASCII 9
AL = 35H (ASCII 5), BL = 39H (ASCII 9)
ADD AL, BL → AL = 6EH (binary result, wrong BCD)
AAA          → Lower nibble E > 9, so:
              AL = 04H (units digit)
              AH = AH + 1 = 01H (tens digit)
```

CF = 1 (decimal carry)
Result: AH:AL = 01:04 = 14 decimal ✓

6.4 AAM – ASCII Adjust after Multiply

Example: 5×9 (unpacked BCD)

AL = 05H (BCD 5), BH = 09H (BCD 9)

MUL BH → AX = 002DH = 45 decimal

AAM → AL = 002DH ÷ 10 = quotient 4, remainder 5

AH = 04H (tens), AL = 05H (units)

AX = 0405H = unpacked BCD for 45 ✓

CHAPTER 7: ASSEMBLER DIRECTIVES

7.1 What are Assembler Directives?

Assembler directives are commands to the assembler (not CPU instructions). They control how the assembler translates your code but do NOT generate machine code instructions.

Directive	Purpose	Example
SEGMENT / ENDSEG	Define start and end of a logical segment	DATA SEGMENT ... DATA ENDS
ASSUME	Tell assembler which segment register maps to which logical segment	ASSUME CS:CODE, DS:DATA, SS:STACK
DB	Define Byte – allocate 1 byte per item	PRICE DB 49H, 98H ;array of 2 bytes
DW	Define Word – allocate 2 bytes per item	NUM DW 1234H ;one word
DD	Define Double Word – allocate 4 bytes	BIG DD 12345678H
DQ	Define Quad Word – allocate 8 bytes (4 words)	HUGE DQ 243598740192A92BH
DT	Define Ten Bytes – allocate 10 bytes	VERY_BIG DT 458733740192A92B8865H
EQU	Give a name to a constant value (like #define in C)	PI EQU 3142 ; use PI in place of 3142
PROC / ENDP	Mark start/end of a procedure (subroutine)	SORT PROC ... SORT ENDP
END	Mark end of entire program module	END START (START = entry point label)
ENDS	Mark end of a segment	DATA ENDS
EVEN	Align next data to even address for faster 16-bit access	EVEN (inserts NOP if needed)
INCLUDE	Insert source code from an external file	INCLUDE MACROS.ASM
OFFSET	Get the offset (displacement) of a variable from segment start	Mov ax, OFFSET PRICES
EXTRN	Declare labels/names defined in another module	EXTRN SORT:NEAR
PUBLIC	Make a label visible to other modules	PUBLIC SORT
ORG	Set the location counter to a specific address	ORG 2000H
MACRO / ENDM	Define a macro (reusable code block with parameters)	MYMACRO MACRO p1,p2 ... ENDM
GROUP	Group multiple segments into one logical group	SYS GROUP CODE, DATA, STACK

7.2 Complete ALP Template (Structure you MUST know)

```
DATA SEGMENT
    NUM1 DB 25H           ; Define byte variable
    NUM2 DB 14H
    RESULT DB ?
DATA ENDS

STACK SEGMENT
    DW 100 DUP (0)        ; Reserve 100 words for stack
STACK ENDS
```

```
CODE SEGMENT
ASSUME CS:CODE, DS:DATA, SS:STACK

START:
    MOV AX, DATA          ; Load data segment address into AX
    MOV DS, AX             ; Initialize DS (CANNOT do MOV DS,DATA directly)

    ; ... your program here ...

    MOV AH, 4CH            ; DOS function: terminate program
    INT 21H               ; Call DOS interrupt

CODE ENDS
END START
```

Why MOV AX, DATA / MOV DS, AX?

You CANNOT directly load a segment register from a constant: MOV DS, DATA is INVALID.

Solution: first load into AX (or any GPR), then copy to DS.

CHAPTER 8: ASSEMBLY LANGUAGE PROGRAMS

8.1 ALP: Average of Two Numbers

```
DATA SEGMENT
    NUM1 DB 20H
    NUM2 DB 30H
    AVG  DB ?
DATA ENDS
CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    MOV AL, NUM1      ; AL = 20H
    MOV AH, 00H       ; Clear AH
    ADD AL, NUM2      ; AL = 20H + 30H = 50H
    ADC AH, 00H       ; AH gets any carry from addition
    MOV BL, 02H
    DIV BL            ; AX / 2; quotient in AL, remainder in AH
    MOV AVG, AL       ; Store result
    MOV AH, 4CH
    INT 21H
CODE ENDS
END START
```

8.2 ALP: Linear Search in Array

```
DATA SEGMENT
    ARR DB 10H, 20H, 30H, 40H, 50H ; Array of 5 bytes
    N   DB 05H                      ; Array size
    KEY DB 30H                      ; Element to search
    POS DB FFH                      ; Position (-1 = not found)
DATA ENDS
CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    MOV CL, N      ; CL = count (5)
    MOV AL, KEY    ; AL = search key
    LEA SI, ARR    ; SI points to start of array
    MOV BL, 01H    ; BL = position counter (1-based)
SEARCH:
    CMP AL, [SI]   ; Compare key with current element
    JE  FOUND      ; Jump if equal
    INC SI         ; Move to next element
    INC BL         ; Increment position
    DEC CL         ; Decrement count
    JNZ SEARCH     ; Loop if count != 0
    JMP NOT_FOUND
FOUND:
    MOV POS, BL    ; Store position
```

```

        JMP DONE
NOT_FOUND:
        MOV POS, FFH          ; FFH indicates not found
DONE:
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

```

8.3 ALP: Palindrome Check

```

DATA SEGMENT
    STR1 DB "MADAM"          ; Input string
    LEN  DB 05H              ; Length of string
    RESULT DB "YES$"         ; Will become "NO$" if not palindrome
DATA ENDS
CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    LEA SI, STR1              ; SI = pointer to start
    MOV CL, LEN
    MOV BL, CL
    DEC BL
    MOV DI, SI
    ADD DI, BX                ; DI = pointer to end (last char)
    MOV CH, 00H
    SHR CX, 1                 ; CX = LEN/2 (compare half)
CHECK:
    MOV AL, [SI]              ; Load char from front
    MOV AH, [DI]              ; Load char from back
    CMP AL, AH                ; Compare
    JNE NOT_PALINDROME
    INC SI
    DEC DI
    LOOP CHECK
    LEA DX, RESULT            ; "YES$"
    JMP DISPLAY
NOT_PALINDROME:
    LEA DX, RESULT
    MOV BYTE PTR[DX], "N"
    MOV BYTE PTR[DX+1], "O"
    MOV BYTE PTR[DX+2], "$"
DISPLAY:
    MOV AH, 09H               ; DOS display string
    INT 21H
    MOV AH, 4CH
    INT 21H
CODE ENDS
END START

```

8.4 ALP: Read & Display Character

```

DATA SEGMENT
    MSG1 DB "Enter a character: $"
    MSG2 DB 0DH, 0AH, "You entered: $"
    MSG3 DB 0DH, 0AH, "INTEL 8086 PROCESSOR$"

```

```

DATA ENDS
CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    LEA DX, MSG1
    MOV AH, 09H           ; Display string
    INT 21H
    MOV AH, 01H           ; Read character into AL
    INT 21H               ; Character stored in AL
    MOV BL, AL            ; Save character in BL
    LEA DX, MSG2
    MOV AH, 09H
    INT 21H
    MOV DL, BL            ; Restore character
    MOV AH, 02H           ; Display character in DL
    INT 21H
    CMP BL, "I"
    JNE DONE
    LEA DX, MSG3
    MOV AH, 09H
    INT 21H
DONE:
    MOV AH, 4CH
    INT 21H
CODE ENDS
END START

```

Key DOS Interrupts (INT 21H) – Must Know!

AH = 01H → Read character from keyboard (result in AL)
 AH = 02H → Display character in DL
 AH = 09H → Display string pointed by DX (must end with \$)
 AH = 0AH → Read string from keyboard into buffer
 AH = 4CH → Terminate program (return to DOS)

CHAPTER 9: IMPORTANT Q & A

9.1 2-Mark Questions with Answers

Q1. What is the purpose of the BIU in 8086?

Ans: The Bus Interface Unit (BIU) handles all communication with the external bus. It fetches instructions from memory into the instruction queue, sends 20-bit physical addresses, and performs all memory/IO read-write operations.

Q2. What is pipelining in 8086?

Ans: Pipelining means the BIU fetches the next instruction from memory while the EU is executing the current instruction. This overlap speeds up execution by keeping both units busy simultaneously.

Q3. What is the physical address formula for 8086?

Ans: Physical Address = Segment Register $\times$ 10H + Effective Address (Offset). The segment register is shifted left by 4 bits (one hex digit) and the EA is added to it.

Q4. Differentiate SHR and SAR.

Ans: SHR (Shift Right Logical): fills MSB with 0, used for unsigned division. SAR (Shift Right Arithmetic): copies sign bit into MSB, used for signed number division. Both shift LSB into CF.

Q5. What is the ASSUME directive?

Ans: ASSUME tells the assembler which logical segment to associate with each segment register (CS, DS, SS, ES). E.g., ASSUME CS:CODE, DS:DATA tells the assembler that the CODE segment holds instructions and DATA holds data.

Q6. What is the role of EQU directive?

Ans: EQU (Equate) assigns a symbolic name to a constant value. The assembler replaces the name with the value wherever it appears. Example: PI EQU 3142 — after this, using PI in the code is the same as using 3142.

Q7. What is segmented memory in 8086?

Ans: Segmented memory divides the 1 MB address space into logical sections called segments. Each segment is up to 64 KB. Four segment registers (CS, DS, SS, ES) hold the base addresses of the code, data, stack, and extra segments respectively.

Q8. What does the Zero Flag (ZF) indicate?

Ans: ZF is set to 1 when the result of an arithmetic or logic operation is zero. It is cleared to 0 when the result is non-zero. It is used in conditional jump instructions like JZ (jump if zero) and JNZ.

Q9. What is the difference between MOV and XCHG?

Ans: MOV copies data from source to destination (source unchanged). XCHG swaps the contents of two operands (both change). Neither instruction affects the flags.

Q10. What is the EVEN directive?

Ans: EVEN tells the assembler to align the next memory allocation to an even address. The 8086 can read a 16-bit word in one bus cycle only if it starts at an even address. A NOP is inserted if the current

address is odd.

9.2 5-Mark Questions with Answers

Q1. Explain the working of DAA with an example.

Answer:

DAA (Decimal Adjust Accumulator) is used after BCD addition to correct the result in AL.

RULE 1: If (lower nibble of AL > 9) OR (AF = 1) → add 06H to AL

RULE 2: If (upper nibble of AL > 9) OR (CF = 1) → add 60H to AL, set CF

Example: 59 BCD + 35 BCD

ADD AL, BL → 59H + 35H = 8EH (binary)

DAA: lower nibble E > 9 → add 06H → 94H

Upper nibble 9 is not > 9, no carry → result = 94 BCD ✓

Example 2: 88 BCD + 49 BCD

ADD AL, BL → D1H, AF=1

DAA: AF=1 → add 06H → D7H; upper D > 9 → add 60H → 37H, CF=1

Result = 137 BCD (CF=1 indicates hundreds)

Q2. List and explain all 9 flags of the 8086 flag register.

Answer:

STATUS FLAGS (set by arithmetic/logic):

1. CF (Carry Flag, bit 0): Set if carry out of MSB or borrow from MSB
2. PF (Parity Flag, bit 2): Set if lower 8 bits have even number of 1s
3. AF (Auxiliary Carry, bit 4): Set if carry from bit 3 to bit 4; used in BCD
4. ZF (Zero Flag, bit 6): Set if result is zero
5. SF (Sign Flag, bit 7): Set if MSB of result is 1 (negative)
6. OF (Overflow Flag, bit 11): Set if signed result overflows the register

CONTROL FLAGS (set by programmer):

7. DF (Direction Flag, bit 10): Controls string operation direction (CLD=auto-inc, STD=auto-dec)
8. IF (Interrupt Flag, bit 9): Enables/disables maskable external interrupts (STI/CLI)
9. TF (Trap Flag, bit 8): Enables single-step mode for debugging

9.3 8/10-Mark Questions with Answer Outlines

Q1. Explain the architecture of the 8086 microprocessor with a block diagram. (10 marks)

Answer points to cover:

- INTRO: 8086 is a 16-bit μ P with 20-bit address bus (1MB), introduced in 1978.
- BLOCK DIAGRAM: Draw two boxes: BIU (left) and EU (right) with buses connecting them.
- BIU components: Instruction Queue (6 bytes), Instruction Pointer (IP), Segment Registers (CS/DS/SS/ES), Address Adder (20-bit physical address generator)
- EU components: ALU (16-bit), Control Unit, Flag Register (9 flags), General Registers (AX, BX, CX, DX), Pointer/Index Registers (SP, BP, SI, DI)
- PIPELINING: BIU fetches while EU executes simultaneously
- BUSES: 20-bit address bus (unidirectional), 16-bit data bus (bidirectional), control bus

- MEMORY: $2^{20} = 1$ MB total addressable memory, divided into 64KB segments
 - CONCLUSION: Segmented architecture allows efficient memory use with 16-bit registers
-

Q2. Explain all addressing modes of 8086 with examples. (8 marks)

Answer points to cover:

- Define addressing mode: how operand location is specified
 - 1. Immediate: data in instruction itself. MOV AX, 5000H
 - 2. Register: operand in register. MOV AX, BX
 - 3. Direct: 16-bit offset in instruction. MOV AX, [2000H]
 - 4. Register Indirect: EA = [BX/SI/DI/BP]. MOV AX, [BX]
 - 5. Based with Displacement: EA = BX/BP + disp. MOV AX, [BX+10H]
 - 6. Indexed with Displacement: EA = SI/DI + disp. MOV AX, [SI+20H]
 - 7. Based Indexed: EA = Base + Index. MOV AX, [BX+SI]
 - 8. Based Indexed + Displacement: EA = Base+Index+Disp. MOV AX, [BX+DI+100H]
 - Physical Address = DS:EA (BX/SI/DI use DS; BP uses SS)
-

Q3. Write an ALP to reverse a string and check if it is a palindrome. (8 marks)

Answer points to cover:

- Initialize DS, load string address into SI, set DI to last char
 - Reverse: swap chars at SI and DI, move SI forward, DI backward, repeat LEN/2 times
 - Compare original with reversed; if all chars match → palindrome
 - Display result using INT 21H, AH=09H (string display)
 - See full code in Section 8.3
-

CHAPTER 10: VIVA QUESTIONS

V1. What is a microprocessor?

→ A microprocessor is a semiconductor IC that contains the CPU (ALU + CU + registers) on a single chip. It fetches, decodes, and executes instructions.

V2. How many address lines does 8086 have? How much memory can it address?

→ 20 address lines. It can address $2^{20} = 1,048,576 = 1 \text{ MB}$ of memory.

V3. What is a segment? Why is segmentation needed?

→ A segment is a logical unit of memory up to 64 KB. Segmentation allows the 16-bit registers to address the full 1 MB memory space by combining a 16-bit segment base with a 16-bit offset.

V4. What is the instruction queue? How large is it?

→ The instruction queue is a 6-byte FIFO buffer in the BIU that holds pre-fetched instruction bytes. It allows the BIU to fetch ahead while the EU executes.

V5. What is the difference between CF and OF?

→ CF (Carry Flag) is for unsigned overflow — carry out of MSB. OF (Overflow Flag) is for signed overflow — result too large/small for signed interpretation.

V6. Which register is used as a counter in LOOP instruction?

→ CX register. LOOP decrements CX and jumps to label if $CX \neq 0$.

V7. What is LEA instruction?

→ Load Effective Address. LEA dest, src loads the offset address of src into dest. E.g., LEA SI, ARR loads the address of ARR into SI (not the contents!).

V8. Why do we write MOV AX, DATA / MOV DS, AX?

→ You cannot load an immediate value directly into a segment register. So the segment address is first loaded into AX (a GPR), then moved to DS.

V9. What is the difference between NEAR and FAR call?

→ NEAR call: procedure in same code segment (only IP pushed). FAR call: procedure in different segment (both CS and IP pushed onto stack).

V10. What does INT 21H, AH=4CH do?

→ Terminates the program and returns control to DOS. It is equivalent to `exit()` in C.

V11. What is the difference between DW and DB?

→ DB (Define Byte) allocates 1 byte per item. DW (Define Word) allocates 2 bytes per item.

V12. Can you use [BP] in direct addressing? What segment does it use?

→ Yes, [BP] is valid for register indirect addressing. It uses the Stack Segment (SS) by default, unlike BX, SI, DI which use DS.

CHAPTER 11: PYQ ANALYSIS & MOST PROBABLE QUESTIONS

11.1 Repeated Questions Analysis

Topic	Times in QB	Predicted Marks	Priority
8086 Block Diagram (Architecture)	5+	10	■ MUST
Physical address calculation	6+	10	■ MUST
Flag register diagram + all 9 flags	4+	6–8	■ MUST
Addressing modes (all types + examples)	4+	8	■ MUST
Shift & Rotate (SHL, SHR, SAR, ROL, ROR)	4+	8	■ MUST
DAA with example	4+	6–8	■ MUST
AAA / AAM with example	3+	6–8	■ HIGH
Assembler directives (ASSUME, EQU, MACRO, DB)	4+	4–8	■ HIGH
ALP: Linear Search	3+	8	■ HIGH
ALP: Palindrome check	2+	8	■ HIGH
MOV, PUSH, POP, IN, XCHG, IMUL	3+	8	■ HIGH
BIU architecture / role	3+	6	■ MEDIUM
Segment registers & segmentation	3+	6–8	■ MEDIUM
ALP: Read char from keyboard	2+	4	■ MEDIUM
Valid/Invalid instruction check	2+	4–6	■ MEDIUM

11.2 Top 20 Most Expected Questions

1. Explain 8086 architecture with block diagram (10 marks)
2. Calculate physical addresses for given instructions and register values
3. Explain all 9 flags of the flag register with a neat diagram
4. List and explain all memory addressing modes with examples
5. Write ALP for linear search in an array of N numbers
6. Write ALP to reverse a string and check palindrome
7. Explain shift (SHL, SHR, SAR) and rotate (ROL, ROR, RCL, RCR) with examples
8. Explain DAA and AAA with worked numerical examples
9. Explain assembler directives: ASSUME, DB, EQU, MACRO, SEGMENT
10. Write ALP to read a character from keyboard and display; if "I", print INTEL 8086 PROCESSOR
11. Explain MOV, PUSH, POP, IN, XCHG, IMUL instructions with examples

12. Explain advantages of segmentation and physical address generation from code segment
13. Explain BIU of 8086 with neat diagram
14. Identify addressing modes and calculate PA for MOV [BX], AX; MOV [BX+DI+3456H], 9ABCH
15. Write ALP using procedures to: read a string, display it, display its length
16. Check if the following 8086 instructions are valid or invalid with reasons
17. Explain AAM (ASCII Adjust after Multiply) with example
18. Explain EVEN, INCLUDE, OFFSET, DD assembler directives
19. Difference between arithmetic and logical shift instructions with examples
20. Explain the role of ASSUME, END, PROC, ENDS directives

CHAPTER 12: SMART REVISION & ONE-NIGHT CHEATSHEET

12.1 Mnemonics & Memory Tricks

Flags order (O D I T S Z A P C):

- "Oh Darn! I Thought Snakes Zapped Any People's Cardigans"

Segment register defaults:

- CS→code, DS→data (BX/SI/DI), SS→stack (SP/BP), ES→string dest (DI)

Base vs Index registers:

- Base: BX, BP (only these two!) | Index: SI, DI (only these two!)

Shift Left = $\times 2$ / Shift Right = $\div 2$:

- Think binary: 0010 = 2; shift left → 0100 = 4 (doubled)

DAA vs DAS vs AAA vs AAS:

- D = packed/Double digit BCD | A = unpacked/ASCII | Addition/Subtraction

PUSH decrements SP, POP increments SP:

- Stack grows DOWN. Push: SP-2. Pop: SP+2.

Physical address = segment $\times$ 10H + offset:

- "shift left 1 hex digit" = multiply by 16 = shift left by 4 bits

12.2 Last-Minute Formula & Fact Sheet

Fact	Value/Rule
Physical Address formula	PA = Segment $\times$ 10H + Offset (EA)
Address bus width	20 bits
Data bus width	16 bits
Max addressable memory	1 MB (2^{20} bytes)

Segment size	Max 64 KB (2^{16} bytes)
Instruction queue size	6 bytes (FIFO)
Number of flags used	9 (out of 16 bits)
Status flags	CF, PF, AF, ZF, SF, OF (6 total)
Control flags	DF, IF, TF (3 total)
Segment registers	CS, DS, SS, ES
Base registers	BX, BP (only these for based addressing)
Index registers	SI, DI (only these for indexed addressing)
[BP] default segment	SS (Stack Segment)
[BX],[SI],[DI] default segment	DS (Data Segment)
INC/DEC and Carry Flag	INC/DEC do NOT affect CF!
MUL byte result	$AX = AL \times src$
MUL word result	$DX:AX = AX \times src$
DIV byte result	Quotient in AL, Remainder in AH
DIV word result	Quotient in AX, Remainder in DX
INT 21H, AH=01H	Read character $\rightarrow$ AL
INT 21H, AH=02H	Display character from DL
INT 21H, AH=09H	Display string (\$-terminated) at DX
INT 21H, AH=4CH	Terminate program
SHL/SAL difference	Identical! Both shift left, fill 0 in LSB
SHR vs SAR	SHR fills 0 in MSB (unsigned); SAR fills sign bit (signed)
ROL vs RCL	ROL: MSB $\rightarrow$ LSB and CF (circular); RCL: CF included in rotation (9-bit)

12.3 Predicted Exam Paper

MICROPROCESSOR & EMBEDDED SYSTEMS — Predicted Paper

Time: 3 Hours | Max Marks: 100 | Answer all questions

Q.No	Question	Marks
1(a)	Explain the architecture of 8086 microprocessor with a neat block diagram, describing the roles of BIU and EU.	
1(b)	Identify addressing modes and calculate physical addresses: i) ADD [DI], AL ii) AND [BX+DI], CL iii) ADC [BP+100H], AX (DS=2000H, SS=4000H, BX=3FFFH, DI=5A00H, BP=69B0H)	10
2(a)	Explain the flag register of 8086 with a neat diagram, describing all 9 flags.	8
2(b)	Write an ALP to search an element in an array using linear search. Display position or "not found".	8

3(a)	Explain shift and rotate instructions (SHL, SHR, SAR, ROL, ROR, RCL, RCR) with syntax and examples.	8
3(b)	Explain DAA and AAA instructions with worked examples.	8
4(a)	Explain the following 8086 instructions with examples: MOV, PUSH, POP, IN, XCHG, IMUL	8
4(b)	Explain assembler directives: ASSUME, DB, DW, EQU, SEGMENT, MACRO, OFFSET, EVEN	8
5(a)	Write an ALP to reverse a string and check if it is a palindrome.	8
5(b)	Explain the concept of segmented memory and how physical address is generated from logical address.	8
6(a)	Write an ALP to read a character from keyboard and display it. If character is "I", print "INTEL 8086 PROCESSOR".	8
6(b)	Explain AAM and DAS with examples. How are they different from AAA and DAA?	6
6(c)	Check if valid/invalid: i) ADD AX,100H ii) MOV BL,AX iii) MOV DS,5000H iv) OUT 9000H,AX v) ADD [5000H],[3000H]	8

12.4 One-Night Before Exam – Revision Priority

- Hours 1–2: Re-draw 8086 block diagram (BIU+EU) with ALL components labelled
- Hour 3: Memorise flag register layout and all 9 flag descriptions
- Hour 4: Practice 3 physical address calculations (direct, based, indexed)
- Hour 5: Memorise addressing mode table (EA formula + example for each)
- Hour 6: Write DAA and AAA examples on paper (do the arithmetic yourself!)
- Hour 7: Study ALP template; trace through linear search ALP line by line
- Hour 8: Quick read of shift/rotate and assembler directives table
- Last 30 min: Read the "Fact Sheet" (Section 12.2) — all key numbers & rules

FINAL EXAM TIPS:

- Always draw diagrams — they earn partial marks even if explanation is incomplete.
- Write the physical address formula at the start of every address calculation question.
- For ALP questions, include DATA SEGMENT, CODE SEGMENT, ASSUME, and INT 21H/4CH.
- For DAA/AAA questions, show binary arithmetic step-by-step.
- For shift questions, show the bit pattern before and after shifting.
- Manage time: 10-mark questions = 15 min max. Move on if stuck.

■ BEST OF LUCK IN YOUR EXAM! You've got this! ■